Voice Synthesis Using Source-Filter Method

By Alex Ryström

February 2026

1 Introduction

The objective of this assignment was to construct a digital model of the human voice based on the Source-Filter model. By generating an idealised glottal source and passing it through a series of cascaded resonant filters (representing the vocal tract), we aimed to synthesise recognisable vowels and join them together into a short, dynamically controlled melody.

2 Methodology: Synthesis Implementation

The synthesis was fully implemented in Python. The method consists of an additive source generator and a filtering stage based on empirical formant data found from the voice lab done on campus.

2.1 The Glottal Source (Additive Synthesis)

The human glottal flow roughly resembles a sawtooth waveform, which contains all integer harmonics dropping off at a rate of -6 dB per octave. To replicate this, an additive synthesis function was constructed using 50 sinusoidal partials. The amplitude of the n-th harmonic had a relationship of $1/n^{\text{spec\_slope}}$. A slope value of 1.0 gives the -6 dB/octave.

To mimic the natural wavering of a human voice, adjustable frequency modulation (vibrato) was implemented as well. The function written generate the source can be seen as follows:

```
def construct_source(f0_note, duration, volume_db, spec_slope, vibrato_freq, vibrato_amplitude):
    t = np.linspace(0.001, duration, int(fsamp * duration), endpoint=False)
    f0 = 440 * 2 ** ((f0_note - 69) / 12) #convert MIDI note to frequency

    f0_vibrato = f0 * (1 + (vibrato_amplitude / t) * np.sin(2 * np.pi * vibrato_freq * t))

    p = np.zeros_like(t)
    for n in range(1, n_partials + 1):
        p += A*(n**(-spec_slope)) * np.sin(2*np.pi * n * f0_vibrato * t)

    p = p / np.max(np.abs(p)) #normalise
    gain = 10 ** (volume_db / 20) #convert dB to linear gain
    return gain * p
```

2.2 The Vocal Tract (Formant Filtering)

The filtering stage shapes the raw source into specific vowels. Five two-pole resonators for different vowels were implemented using the equations from the JoLi filters cheat sheet given in the assignment. The filter relies on specified formant frequencies (F) and bandwidths (B) to calculate the Quality

factor ($Q = F/B$). The filter coefficients a_1 and a_2, along with a normalisation gain factor G, were calculated and applied using scipy.signal.lfilter.

2.3 Dynamic Control

Vocal dynamics naturally alter both the volume and the brightness of the voice. This was implemented via two control parameters:

1. **Sound Level (dB):** A gain multiplier applied after the source generation.
2. **Spectral Slope:** Altering the exponent on the partial amplitudes. A smaller slope (e.g., < 1.0) simulates a louder voice by keeping more higher-frequency energy, while a higher slope simulates a softer, more dark voice.

3 Testing and Spectrogram Comparison

A test melody consisting of five notes (C3, D3, E3, F3, G3) and a changing vowel sequence (`[a] -> [oe] -> [i] -> [u] -> [a2]`) was generated (a2 being a different take of /a/ from the lab).

To evaluate the how "natural" the synthesis was, two spectrograms were plotted showing the synthesised melody alongside a recreated home recording of the same melodic phrase.

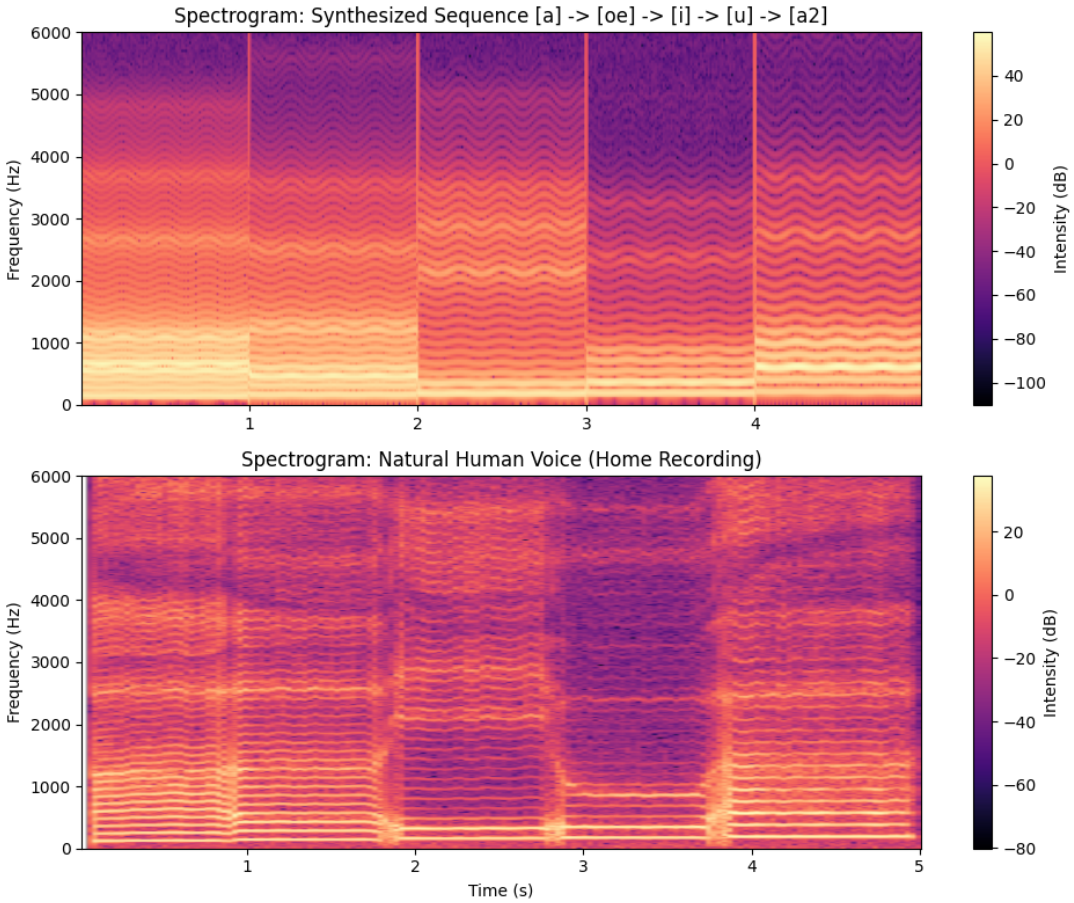

Figure 1: Spectrogram comparison of the synthesised melody (top) and natural human voice (bottom).

3.1 Observations from the Spectrograms

- **Formant Transitions:** In the synthesised plot, the formants appear as stiff blocks that jump instantaneously when the notes change. In the natural recording, these bands transition smoothly into one another as the jaw, tongue, and lips move.
- **Stability:** The synthesised harmonics are evenly spaced, even during vibrato. The natural recording has an irregularity on the horizontal lines that show the instability of real vocal folds.
- **Breathiness:** Especially visible in the higher frequencies, the synthesised spectrogram is darker between the partials. The natural recording is slightly cloudy in appearance likely due to the person's breath.

4 Missing Features

While the synthesis is good enough for the pitch and vowel to be identified, it sounds "robotic". Some problems and areas of improvement could be in:

4.1 Perfect Phase

Additive synthesis makes it so every partial is perfectly locked in phase. A real glottal pulse has complex phase relationships and variations producing what we identify as a real voice. The perfection of the generated source is interpreted by the ear as an electronic buzz rather than someone speaking.

4.2 Lack of Turbulent Noise

Without a noise component to simulate air moving through the vocal folds (especially in softer dynamics), the voice lacks "breathiness" and feels unnatural.

4.3 Static Filter Coefficients

Updating the filter coefficients abruptly at the start of each new note causes instantaneous shifts in sound, entirely missing out on the gliding of speech that we perceive as natural.

5 Suggested Improvements

To further improve the synthesis from robotic to a more convincing human voice, the following improvements to the code and theory could possibly be made:

- **Coefficient Adjusting:** Instead of assigning static formant frequencies per note, the a_1 and a_2 filter coefficients could be linearly adjusted over a few milliseconds during note transitions to create more realistic changes in vowels.
- **Adding Noise:** Mixing a white noise signal into the glottal source before applying the formant filters could add the turbulence that is missing.

6 Conclusion

The additive source and cascaded two-pole filter model that was implemented is quite effective at recreating a voice, especially for using only very fundamental concepts, allowing us to control the note, vowel, and dynamics in the sound. However, observation of the two spectrograms comparing a real voice vs the synthesised one shows that a truly natural sounding voice requires the inclusion of "imperfections", such as shimmer, jitter, and continuous transitions that basic synthesis fails to capture.